

Fiche 309 — Écrire ses fonctions : arguments, défauts, ... et évaluation paresseuse

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 10 « Writing your own functions » (§10.1 exemples simples, §10.2 opérateurs binaires, §10.3 arguments nommés et défauts, §10.4 l'argument <code>...</code> , §10.5 assignations dans une fonction, §10.6 exemples avancés)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, chapitre 4 « Functions » — §4.1 écrire, §4.1.2 arguments, §4.2 fonctions comme objets, §4.3.1 environnement d'évaluation, §4.3.2 appariement des arguments , §4.3.3 évaluation des arguments ; §2.1.5 « Function objects », §2.1.8 « Promise objects »
Difficulté	Avancé — le chapitre qui fait passer d'utilisateur à programmeur
Temps d'étude estimé	2 h
Prérequis	Fiches 300 à 308
Concepts clés	définition, valeur = dernière expression , fonction anonyme , opérateur <code>%nom%</code> , arguments nommés et positionnels , défauts arbitraires , <code>...</code> , <code>list(...)</code> , <code>..1 / ..n</code> , assignations locales , <code><<-</code> , <code>assign()</code> , appariement en trois passes , <code>missing()</code> , call-by-value , évaluation paresseuse , promesses , forcer , <code>substitute()</code> , <code>match.arg</code> , <code>match.call</code> , <code>pmatch</code>
À retenir en priorité	L'appariement en trois passes · les défauts sont évalués dans la fonction, les arguments fournis chez l'appelant · l'évaluation paresseuse · <code><<-</code> · le fait qu'une fonction ne modifie jamais son argument.

Vue d'ensemble

DEFINIR	<code>nom <- function(arg1, arg2, ...) expression</code> « The value of the expression is THE VALUE RETURNED » une fonction sans nom est ANONYME (apply, outer...)
OPERATEUR	<code>"%!" <- function(X, y) { ... }</code> s'utilise <code>X %! y</code> (noter les GUILLEMETS dans la définition)

ARGUMENTS positionnels, puis nommes -- dans N'IMPORTE QUEL ORDRE une fois nommes
defaults : arg = expression
« defaults may be ARBITRARY EXPRESSIONS, even involving OTHER
ARGUMENTS to the same function »

APPARIEMENT EN TROIS PASSES

1. tags EXACTS
 2. tags PARTIELS (seulement AVANT le ... s'il y en a un)
 3. POSITION (le ... prend tout le reste, nomme ou non)
- reste non apparie -> ERREUR

... passe des arguments a une autre fonction
list(...) les evalue tous ; ..1, ..2, ..n un par un

ASSIGNATIONS « any ordinary assignments done within the function are
LOCAL AND TEMPORARY and are LOST AFTER EXIT »
pour sortir : <<- ou assign()

EVALUATION call-by-value : changer un argument NE CHANGE PAS l'appelant
PARESSEUSE : « arguments are NOT EVALUATED UNTIL NEEDED »
« in some cases the argument WILL NEVER BE EVALUATED »
une PROMESSE : expression + environnement -> puis valeur, une SEULE fois

OU SONT-ILS EVALUES ?

arguments FOURNIS -> dans le cadre de l'APPELANT
arguments par DEFAUT -> dans le cadre de la FONCTION

Le problème posé. « *The R language allows the user to **create objects of mode function. These are true R functions** that are stored in a special internal form and may be used in further expressions ... **In the process, the language gains enormously in power, convenience and elegance, and learning to write useful functions is one of the main ways to make your use of R comfortable and productive.*** » (§10)

INTUITION — IL N'Y A PAS DEUX CATÉGORIES DE FONCTIONS.

« *It should be emphasized that most of the functions supplied as part of the R system, such as `mean()` , `var()` , `postscript()` and so on, are themselves written in R and thus do not differ materially from user written functions.* » (§10)

Le §1.1 l'avait annoncé (« *most of the system supplied functions are themselves written in the S language* », fiche 300). Ici, la conséquence est explicitée : **vos fonctions sont des citoyennes de première classe**. Elles se passent en argument, se rangent dans des listes, se renvoient comme résultat — et rien ne les distingue de `mean` .

● Concept 1 — Définir une fonction

DÉFINITION (§10).

« *A function is defined by an assignment of the form*

```
nom <- function(arg_1, arg_2, ...) expression
```

The `expression` is an R expression, (***usually a grouped expression***), that uses the arguments to calculate a value. ***The value of the expression is the value returned for the function.*** »

« *A call to the function then usually takes the form `nom(expr_1, expr_2, ...)` and **may occur anywhere a function call is legitimate.*** »

Il n'y a pas de `return` obligatoire. La valeur de la fonction est **celle de son corps**, donc celle de sa **dernière expression** (fiche 308, §9.1 : « *the value of the group is the result of the last expression* »).

COMPLÉMENT (R LANGUAGE DEFINITION §4.1.1).

*« The syntax for writing a function is `function ( arglist ) body . ...` An argument list is a **comma separated list of formal arguments**. A formal argument can be a **symbol**, a **statement of the form** `symbol = expression` , or **the special formal argument** `... .` »*

« *Generally functions are assigned to symbols but they don't need to be. The value returned by the call to `function` is a function. If this is not given a name it is referred to as an anonymous function. Anonymous functions are most frequently used as arguments to other functions such as the `apply` family or `outer` .* »

C'est exactement ce qu'on a écrit en fiche 305 sans le nommer :

```
z <- outer(x, y, function(x, y) cos(y)/(1 + x^2))
```

La fonction n'a **pas de nom** : elle est créée, passée, utilisée, oubliée.

COMPLÉMENT (§4.2).

« *Functions are first class objects in R. They can be used anywhere that an R object is required. In particular they can be passed as arguments to functions and returned as values from functions.* »

1.1 Le premier exemple du cours : la statistique t à deux échantillons

Exemple (§10.1). *« consider a function to calculate the two sample t-statistic, **showing "all the steps"**. This is an **artificial example**, of course, since there are other, simpler ways of achieving the same end. »*

```
twosam <- function(y1, y2) {  
  n1 <- length(y1); n2 <- length(y2)  
  yb1 <- mean(y1); yb2 <- mean(y2)  
  s1 <- var(y1); s2 <- var(y2)  
  s <- ((n1-1)*s1 + (n2-1)*s2)/(n1+n2-2)  
  tst <- (yb1 - yb2)/sqrt(s*(1/n1 + 1/n2))  
}
```

```
tst  
}
```

```
tstat <- twosam(data$male, data$female); tstat
```

CONTRÔLE — RECONNAÎTRE LA FORMULE.

La variable `s` est la **variance mise en commun** (*pooled variance*)

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

et `tst` est la statistique de Student à variances égales

$$t = \frac{\bar{y}_1 - \bar{y}_2}{s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

— le dénominateur du code, `sqrt(s*(1/n1 + 1/n2))`, est bien $s_p \sqrt{1/n_1 + 1/n_2}$ puisque `s` est la **variance**, pas l'écart-type.

Et la dernière ligne, `tst`, **n'est pas une coquetterie** : c'est ce qui fait la valeur de retour.

1.2 Le second exemple : le « `backslash` » de Matlab

Exemple (§10.1). *« consider a function to **emulate directly the Matlab backslash command**, which returns **the coefficients of the orthogonal projection of the vector `y` onto the column space of the matrix `X`**. (This is ordinarily called the least squares estimate of the regression coefficients.) »*

« given a $n \times 1$ vector `y` and an $n \times p$ matrix `X`, then $X \backslash y$ is defined as $(X^T X)^{-1} X^T y$, where $(X^T X)^{-1}$ is a **generalized inverse** ».

```
bslash <- function(X, y) {  
  X <- qr(X)  
  qr.coef(X, y)  
}
```

```
}  
regcoeff <- bslass(Xmat, yvar)
```

« The classical R function `lsfit()` **does this job quite well, and more. It in turn uses the functions `qr()` and `qr.coef()` in the slightly counterintuitive way above. Hence there is probably some value in having just this part isolated in a simple to use function if it is going to be in frequent use.** »

MÉTHODE — LA JUSTIFICATION D'ÉCRIRE UNE FONCTION.

Le cours ne dit pas « écrivez des fonctions parce que c'est bien ». Il donne un critère : **isoler un usage fréquent d'une mécanique contre-intuitive**. `qr()` puis `qr.coef()` est « *slightly counterintuitive* » ; l'envelopper une fois évite de s'en souvenir vingt fois.

Notez `x <- qr(X)` dans le corps. L'argument `x` est réassigné — et le §10.5 garantit que **cela n'affecte pas l'appelant**. C'est ce qui rend l'écriture sûre.

● Concept 2 — Définir un opérateur binaire

MÉTHODE (§10.2).

*« Had we given the `bslass()` function a different name, namely **one of the form `%anything%`**, it **could have been used as a binary operator** in expressions rather than in function form. »*

```
"%!%" <- function(X, y) { ... }    # noter les GUILLEMETS  
X %!% y                          # s'utilise ainsi
```

« (Note the use of quote marks.) ... The backslash symbol itself is not a convenient choice as it presents special problems in this context. »

« The matrix multiplication operator `%*%` and the outer product matrix operator `%o%` are other examples of binary operators defined in this way. »

EN CLAIR — POURQUOI LES GUILLEMETS.

`%!%` n'est pas un identifiant valide (fiche 300) : le parseur ne peut pas le lire comme un nom. Les guillemets en font **une chaîne**, et l'assignation crée un objet portant ce nom. C'est le mécanisme de la fiche 302 : « *objects can have names that are not identifiers* ». Et à l'usage, c'est la remarque du §3.1.4 (fiche 301) qui joue : **un opérateur est une fonction**, donc définir une fonction au bon nom suffit à créer un opérateur.

● Concept 3 — Arguments nommés et valeurs par défaut

Règle (§10.3). « *if arguments to called functions are given in the "name=object" form, they may be given in any order. Furthermore the argument sequence may begin in the unnamed, positional form, and specify named arguments after the positional arguments.* »

```
fun1 <- function(data, data.frame, graph, limit) { ... }

ans <- fun1(d, df, TRUE, 20)                # tout positionnel
ans <- fun1(d, df, graph = TRUE, limit = 20) # mixte
ans <- fun1(data = d, limit = 20, graph = TRUE, data.frame = df) # tout nommé, ordre libre
```

« *are all equivalent* ».

Règle — les défauts (§10.3). « *In many cases arguments can be given **commonly appropriate default values**, in which case they may be omitted altogether from the call when the defaults are appropriate.* »

```
fun1 <- function(data, data.frame, graph = TRUE, limit = 20) { ... }

ans <- fun1(d, df)                # équivalent aux trois appels ci-dessus
ans <- fun1(d, df, limit = 10)    # « changes one of the defaults »
```

La phrase à ne pas manquer (§10.3). « *It is important to note that defaults may be arbitrary expressions, even involving other arguments to the same function ; they are not restricted to be constants as in our simple example here.* »

Un défaut peut donc s'écrire `n = length(x)` , `label = deparse(x)` , `sd = sqrt(var(x))` . Cela n'est possible que parce que **les défauts ne sont pas évalués à la définition**, mais **au moment où on en a besoin** — c'est le concept 7.

COMPLÉMENT (R LANGUAGE DEFINITION §4.1.2).

« *Default values ... can be specified using the special form `name = expression` . In this case, **if the user does not specify a value ... the expression will be associated with the corresponding symbol. When a value is needed the expression is evaluated in the evaluation frame of the function.*** »

*« Default behaviours can also be specified by using **the function `missing`** . **When `missing` is called with the name of a formal argument it returns `TRUE` if the formal argument was not matched with any actual argument and has not been subsequently modified in the body of the function. ... The `missing` function does not force evaluation of the argument.** »*

● Concept 4 — L'argument ...

DÉFINITION (§10.4).

« *Another frequent requirement is **to allow one function to pass on argument settings to another.** For example many graphics functions use the function `par()` and functions like `plot()` **allow the user to pass on graphical parameters to `par()` to control the graphical output.** This can be done by **including an extra argument, literally `...` , of the function, which may then be passed on.*** »

```
fun1 <- function(data, data.frame, graph = TRUE, limit = 20, ...) {  
  [omitted statements]  
  if (graph)  
    par(pch = "*", ...)
```



```
[more omissions]
}
```

Règle (§10.4). « *Less frequently, a function will need to refer to components of `...`. The expression `list(...)` evaluates all such arguments and returns them in a named list, while `..1`, `..2`, etc. evaluate them one at a time, with `..n` returning the n-th unmatched argument.* »

Besoin	Écriture
Transmettre tels quels	mettre <code>...</code> dans l'appel — c'est l'usage normal
Les examiner tous	<code>list(...)</code> — les évalue et rend une liste nommée
En prendre un seul	<code>..1</code> , <code>..2</code> , ... — le n-ième non apparié

COMPLÉMENT (R LANGUAGE DEFINITION §4.1.2).

« *The special type of argument `...` can contain any number of supplied arguments. It is used for a variety of purposes. It allows you to write a function that takes an arbitrary number of arguments. It can be used to absorb some arguments into an intermediate function which can then be extracted by functions called subsequently.* »

Et (§2.1.9, fiche 303) : « *If a function has `...` as a formal argument then any actual arguments that do not match a formal argument are matched with `...`.* »

`list(...)` force l'évaluation. Le §2.1.9 le note entre parenthèses : « *The object can be captured (with promises being forced!) as a list* »*. Tant qu'on se contente de transmettre `...`, rien n'est évalué (concept 7) ; dès qu'on écrit `list(...)`, tout l'est.

● Concept 5 — L'appariement des arguments en trois passes

Règle (R Language Definition §4.3.2). « *The first thing that occurs in a function evaluation is the matching of formal to the actual or supplied arguments. This is done by a three-pass process* » :

Passe	Règle
1. Tags exacts	« For each named supplied argument the list of formal arguments is searched for an item whose name matches exactly . It is an error to have the same formal argument match several actuals or vice versa. »
2. Tags partiels	« Each remaining named supplied argument is compared to the remaining formal arguments using partial matching ... It is an error to have multiple partial matches. ... If the formal arguments contain ... then partial matching is only applied to arguments that precede it. »
3. Position	« Any unmatched formal arguments are bound to unnamed supplied arguments, in order . If there is a ... argument, it will take up the remaining arguments, tagged or not. »

« **If any arguments remain unmatched an error is declared.** »

L'exemple du manuel, à décortiquer (§4.3.2). *« Notice that if `f <- function(fumble, foey)` `fbody` , then `f(f = 1, fo = 2)` **is illegal**, even though **the 2nd actual argument only matches foey** . `f(f = 1, foey = 2)` **is legal though**, since **the second argument matches exactly and is removed from consideration for partial matching.** »*

Pourquoi `f(f = 1, fo = 2)` échoue. Passe 1 : aucun tag n'apparie exactement (`f` n'est pas `fumble` , `fo` n'est pas `foey`). Passe 2 : `f` est un préfixe **de `fumble` et de `foey`** → **appariement partiel multiple** → erreur. Le fait que `fo` désigne sans ambiguïté `foey` **n'y change rien**, parce que `f` a été traité **avant**.

Pourquoi `f(f = 1, foey = 2)` réussit. Passe 1 : `foey` apparie **exactement** et **sort du jeu**. Passe 2 : `f` n'a plus que `fumble` comme candidat → appariement partiel **unique** → succès.

La leçon : l'appariement partiel dépend de **ce qui reste**, donc de l'ordre des passes. Un argument abrégé peut devenir ambigu **parce qu'un autre argument a changé**.

L'exception des primitives (§4.3.2). *« This subsection applies to closures but not to **primitive functions**. The latter **typically ignore tags and do positional matching**, but their help pages should be consulted **for exceptions, which include** `log` , `round` , `signif` , `rep` **and** `seq.int` . »*

*« Argument matching is augmented by the functions `match.arg` , `match.call` **and** `match.fun` . Access to the partial matching algorithm used by R is via `pmatch` . »*

● Concept 6 — Les assignations dans une fonction sont locales

Règle (§10.5). « *Note that any ordinary assignments done within the function are local and temporary and are lost after exit from the function. Thus the assignment `X <- qr(X)` does not affect the value of the argument in the calling program.* »

COMPLÉMENT (R LANGUAGE DEFINITION §4.3.3).

« *The semantics of invoking a function in R are call-by-value. In general, supplied arguments behave as if they are local variables initialized with the value supplied and the name of the corresponding formal argument. Changing the value of a supplied argument within a function will not affect the value of the variable in the calling frame.* »

C'est la sémantique de copie de la fiche 302, appliquée aux fonctions. **Une fonction R ne modifie jamais son argument** — la seule exception du langage restant l'environnement (fiche 303).

MÉTHODE — SORTIR DE LA FONCTION QUAND MÊME (§10.5).

« *If global and permanent assignments are intended within a function, then either the "superassignment" operator, `<<-`, or the function `assign()` can be used.* »

« *To understand completely the rules governing the scope of R assignments the reader needs to be familiar with the notion of an evaluation frame. This is a somewhat advanced, though hardly difficult, topic and is not covered further here.* » — c'est l'objet de la **fiche 310**.

● Concept 7 — Évaluation paresseuse et promesses

C'est la partie du langage qui explique le plus de comportements surprenants, et *An Introduction to R* n'en dit rien. Tout vient de la *R Language Definition* §4.3.3.

Règle — où les arguments sont évalués. « *One of the most important things to know about the evaluation of arguments to a function is that supplied arguments and default arguments are treated differently. The supplied arguments to a function are evaluated in the evaluation frame of the calling function. The default arguments to a function are evaluated in the evaluation frame of the function.* »

	Évalué où
Argument fourni par l'appelant	dans le cadre de l'appelant
Argument par défaut	dans le cadre de la fonction

C'est ce qui rend possible la phrase du §10.3 : « *defaults may be arbitrary expressions, even involving other arguments to the same function* ». Un défaut `n = length(x)` est évalué là où `x` existe — à l'intérieur de la fonction.

Règle — la paresse. « *R has a form of lazy evaluation of function arguments. Arguments are not evaluated until needed. It is important to realize that in some cases the argument will never be evaluated.* »

La conséquence de style (§4.3.3). « *Thus, it is bad style to use arguments to functions to cause side-effects. While in C it is common to use the form `foo(x = y)` to invoke `foo` with the value of `y` and simultaneously to assign the value of `y` to `x`, this same style should not be used in R. There is no guarantee that the argument will ever be evaluated and hence the assignment may not take place.* »

« It is also worth noting that the effect of `foo(x <- y)`, if the argument is evaluated, is to change the value of `x` in the calling environment and not in the evaluation environment of `foo`. »

7.1 Le mécanisme : la promesse

DÉFINITION (§4.3.3).

« It is possible to **access the actual (not default) expressions used as arguments inside the function. The mechanism is implemented via promises.** When a function is being evaluated **the actual expression used as an argument is stored in the promise together with a pointer to the environment the function was called from.** When (if) the argument is evaluated **the stored expression is evaluated in the environment that the function was called from. ... The resulting value is then also stored in a separate spot in the promise. Subsequent evaluations retrieve this stored value (a second evaluation is not carried out).** Access to the unevaluated expression is also available using `substitute`. »

Le détail complet :

« When a function is called, **each formal argument is assigned a promise in the local environment of the call, with the expression slot containing the actual argument (if it exists) and the environment slot containing the environment of the caller.** If no actual argument ... is given ... and there is a default expression, it is similarly assigned to the expression slot, but with the environment set to the local environment. »

« The process of filling the value slot of a promise by evaluating the contents of the expression slot in the promise's environment is called **forcing the promise.** A promise will only be forced **once**, the value slot content being used directly later on. »

Résumé du mécanisme en quatre temps : (1) à l'appel, chaque argument formel reçoit **une promesse** ; (2) la promesse contient **l'expression** et **l'environnement** ; (3) au premier usage, elle est **forcée** — l'expression est évaluée **dans son environnement** ; (4) la valeur est **mémorisée**, et les usages suivants la relisent.

L'exemple du manuel — pourquoi une ligne « inutile » ne l'est pas (§4.3.3). « This is occasionally useful when **a default expression depends on the value of another formal argument or other variable in the local environment.** This is seen in the following example **where the lone `Label` ensures that the label is based on the value of `x` before it is changed in the next line** » :

```
function(x, label = deparse(x)) {
  label          # <- FORCE la promesse MAINTENANT
  x <- x + 1
  print(label)
}
```

Sans la ligne `label` seule, la promesse ne serait forcée qu'au `print`, donc **après** `x <- x + 1` — et `deparse(x)` décrirait la valeur modifiée. La ligne isolée ne « fait rien » : elle **fixe le moment de l'évaluation**.

« *The expression slot of a promise can itself involve other promises. This happens whenever an unevaluated argument is passed as an argument to another function. **When forcing a promise, other promises in its expression will also be forced recursively** as they are evaluated.* »

● Concept 8 — Trois exemples avancés du cours

Énoncé. « A block design is defined by two factors, say `blocks` (`b` levels) and `varieties` (`v` levels). If R and K are the $v \times v$ and $b \times b$ **replications and block size matrices**, and N is the $b \times v$ **incidence matrix**, then the efficiency factors are defined as **the eigenvalues of the matrix**

$$E = I_v - R^{-1/2} N^T K^{-1} N R^{-1/2} = I_v - A^T A, \quad A = K^{-1/2} N R^{-1/2}$$

```
bdeff <- function(blocks, varieties) {
  blocks    <- as.factor(blocks)          # minor safety move
  b         <- length(levels(blocks))
  varieties <- as.factor(varieties)       # minor safety move
  v         <- length(levels(varieties))
  K <- as.vector(table(blocks))           # remove dim attr
  R <- as.vector(table(varieties))        # remove dim attr
  N <- table(blocks, varieties)
  A <- 1/sqrt(K) * N * rep(1/sqrt(R), rep(b, v))
  sv <- svd(A)
  list(eff = 1 - sv$d^2, blockcv = sv$u, varietycv = sv$v)
}
```

Étape 1 — les « *minor safety moves* ». `as.factor()` sur les deux arguments : la fonction accepte donc **aussi bien un facteur qu'un vecteur ordinaire**. C'est le même mécanisme que celui noté au §4.2 pour `tapply` (fiche 304) — « *arguments are coerced to factors when necessary* ».

Étape 2 — les « *remove dim attr* ». `table()` rend **un tableau à une dimension** (fiche 302), muni d'un `dim`. `as.vector()` l'enlève — c'est la voie officielle du §5.9 (fiche 305). Sans cela, l'arithmétique de l'étape 4 se heurterait à la règle 3 du recyclage mixte : « *the arrays must all have the same `dim` attribute or an error results* ».

Étape 3 — l'incidence. `N <- table(blocks, varieties)` — la voie directe du §5.3 (fiche 305), préférée à `crossprod` des indicatrices.

Étape 4 — la ligne dense. `A <- 1/sqrt(K) * N * rep(1/sqrt(R), rep(b, v))` construit $K^{-1/2}NR^{-1/2}$ **par recyclage** : le premier facteur, de longueur b , se recycle **le long des colonnes** de la matrice $b \times v$ (ordre colonne-majeur, fiche 305) ; le second, fabriqué par `rep(1/sqrt(R), rep(b, v))`, répète **chaque** élément de $R^{-1/2}b$ **fois**, ce qui le fait varier **par colonne**. Multiplier à gauche par une diagonale et à droite par une autre, **sans construire aucune matrice diagonale**.

Étape 5 — la SVD plutôt que les valeurs propres. « *It is **numerically slightly better** to work with the singular value decomposition on this occasion rather than the eigenvalue routines.* » Puisque $E = I_v - A^T A$, les valeurs propres de E valent $1 - d_i^2$ où les d_i sont **les valeurs singulières de A** — d'où `1 - sv$d^2`. Calculer $A^T A$ puis ses valeurs propres élèverait le conditionnement au carré ; la SVD l'évite.

Étape 6 — le résultat est une liste. « *The result of the function is **a list giving not only the efficiency factors as the first component, but also the block and variety canonical contrasts, since sometimes these give additional useful qualitative information.*** » C'est le patron du §1.3 (fiche 300) : **rendre un objet interrogeable**, pas un nombre.

Énoncé. « *For printing purposes with large matrices or arrays, it is often useful to **print them in close block form without the array names or numbers. Removing the `dimnames` attribute will not achieve this effect, but rather the array must be given a `dimnames` attribute consisting of empty strings.*** »

La version manuelle :

```
temp <- X
dimnames(temp) <- list(rep("", nrow(X)), rep("", ncol(X)))
```

```
temp; rm(temp)
```

La version en fonction :

```
no.dimnames <- function(a) {  
  ## Remove all dimension names from an array for compact printing.  
  d <- list()  
  l <- 0  
  for (i in dim(a)) {  
    d[[l <- l + 1]] <- rep("", i)  
  }  
  dimnames(a) <- d  
  a  
}
```

Étape 1 — pourquoi `dimnames(a) <- NULL` ne suffit pas. Sans `dimnames`, R **réinvente** des étiquettes `[1,]`, `[,1]` ... à l'affichage. Pour n'avoir **rien**, il faut des noms qui soient **des chaînes vides** — c'est le point que le cours souligne.

Étape 2 — la boucle. Elle parcourt `dim(a)`, donc fonctionne pour **un nombre quelconque de dimensions**, pas seulement pour une matrice.

Étape 3 — la ligne à décoder. `d[[l <- l + 1]] <- rep("", i)`. L'assignation `l <- l + 1` est **une expression qui vaut la nouvelle valeur** (fiche 308, §9.1) : on incrémente **et** on utilise le résultat comme indice, en une fois. C'est l'illustration directe de « *an assignment is an expression whose result is the value assigned* ».

Étape 4 — la valeur de retour. Le `a` final. La fonction **modifie sa copie locale** et la rend ; l'original de l'appelant est intact (§10.5).

Étape 5 — la remarque du cours. « It also **illustrates how some effective and useful user functions can be quite short.** » Et : « This is particularly useful for **large integer arrays, where patterns are the real interest rather than the values.** »

Énoncé. « *Functions may be recursive, and may themselves define functions within themselves. Note, however, that such functions, or indeed variables, are not inherited by called functions in higher evaluation frames as they would be if they were on the search path.* »

« *The integrand is evaluated at **the end points of the range and in the middle**. If the one-panel trapezium rule answer is close enough to the two panel, then the latter is returned as the value. Otherwise the same process is recursively applied to each panel. The result is an adaptive integration process that concentrates function evaluations in regions where the integrand is farthest from linear.* »

```
area <- function(f, a, b, eps = 1.0e-06, lim = 10) {  
  fun1 <- function(f, a, b, fa, fb, a0, eps, lim, fun) {  
    ## function `fun1` is only visible inside `area`  
    d <- (a + b)/2  
    h <- (b - a)/4  
    fd <- f(d)  
    a1 <- h * (fa + fd)  
    a2 <- h * (fd + fb)  
    if (abs(a0 - a1 - a2) < eps || lim == 0)  
      return(a1 + a2)  
    else {  
      return(fun(f, a, d, fa, fd, a1, eps, lim - 1, fun) +  
             fun(f, d, b, fd, fb, a2, eps, lim - 1, fun))  
    }  
  }  
  fa <- f(a)  
  fb <- f(b)  
  a0 <- ((fa + fb) * (b - a))/2  
  fun1(f, a, b, fa, fb, a0, eps, lim, fun1)  
}
```

Étape 1 — la règle du trapèze. Sur $[a, b]$, un panneau donne $a_0 = \frac{(f(a)+f(b))(b-a)}{2}$. Deux panneaux, avec d le milieu et $h = (b - a)/4$, donnent $a_1 = h(f(a) + f(d))$ et $a_2 = h(f(d) + f(b))$ — chacun étant l'aire d'un trapèze de largeur $(b - a)/2$, d'où le h qui vaut **le quart** de la largeur totale.

Étape 2 — le critère d'arrêt. `abs(a0 - a1 - a2) < eps` compare **une estimation grossière à une estimation fine**. Si elles s'accordent, la fine est acceptée. C'est le principe de l'intégration **adaptative**.

Étape 3 — le garde-fou. `|| lim == 0` borne la profondeur de récursion. Et `||` **est ici indispensable** : c'est l'opérateur **court-circuit** de la fiche 308.

Étape 4 — le point le plus déroutant : pourquoi `fun` en argument ? `fun1` s'appelle elle-même **sous le nom `fun`**, qu'elle reçoit en argument, et `area` termine par `fun1(..., fun1)` — elle **se passe elle-même**. C'est là qu'intervient l'avertissement du cours : « *such functions ... **are not inherited by called functions in higher evaluation frames as they would be if they were on the search path*** ». `fun1` est **locale à `area`** ; les cadres d'évaluation créés par ses propres appels **ne la voient pas** par le chemin de recherche. La passer en argument est **la solution que le cours retient**. C'est « *the little puzzle in R programming* » qu'il annonce.

Étape 5 — la mise en garde de performance. « *There is, however, **a heavy overhead**, and the function is **only competitive with other algorithms when the integrand is both smooth and very difficult to evaluate***. » Le cours ne présente donc **pas** cette fonction comme un bon intégrateur, mais comme un exercice de programmation.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« écrire une fonction qui... »	<code>nom <- function(args) { ... }</code> — dernière expression = valeur
« la passer à <code>apply</code> , <code>outer</code> , <code>sapply</code> »	une fonction anonyme suffit
« en faire un opérateur »	<code>"%nom%" <- function(a, b) ...</code>
« appeler dans n'importe quel ordre »	arguments nommés
« une valeur par défaut dépendant d'un autre argument »	c'est permis — évalué dans la fonction
« transmettre des options à <code>par()</code> »	<code>...</code>
« un nombre quelconque d'arguments »	<code>...</code> , puis <code>list(...)</code>
« le n-ième argument non apparié »	<code>..n</code>
« erreur : argument apparié plusieurs fois »	appariement partiel ambigu — passe 2
« mon abréviation d'argument marchait avant »	l'ambiguïté dépend de ce qui reste
« ma fonction ne modifie pas la variable »	call-by-value — c'est normal
« je veux modifier une variable extérieure »	<code><<-</code> ou <code>assign()</code>
« l'argument invalide ne provoque pas d'erreur »	il n'a jamais été évalué
« <code>deparse(x)</code> décrit la mauvaise valeur »	forcer la promesse plus tôt
« savoir si l'argument a été fourni »	<code>missing()</code> — sans forcer l'évaluation
« récupérer le texte de l'argument »	<code>substitute()</code>
« une fonction récursive locale »	la passer en argument — elle n'est pas héritée

Comment résoudre ce type d'exercice

Protocole « écrire une fonction utilisable » — 5 étapes.

1. **Identifier ce qui varie** — cela devient les arguments ; ce qui ne varie presque jamais devient **un défaut**.
2. **Ordonner les arguments** : les indispensables d'abord (appelés positionnellement), les optionnels ensuite (appelés par nom).
3. **Placer ...** si la fonction transmet à une autre — et se rappeler qu'après lui, **l'appariement partiel ne joue plus**.
4. **Rendre un objet, pas un nombre**, quand plusieurs choses sont calculées : une **liste nommée** (patron `bdeff`).
5. **Ne pas oublier la dernière expression** — c'est la valeur de retour ; `return()` n'est nécessaire que pour **sortir tôt**.

Protocole « déboguer un appariement d'arguments » — 4 étapes.

1. Écrire les **noms complets** des arguments : si l'erreur disparaît, c'était un **appariement partiel**.
2. Vérifier la **passé 1** : un même formel apparié deux fois → erreur.
3. Vérifier la **passé 2** : un préfixe ambigu → erreur, **même si un autre argument le désambiguiserait**.
4. Vérifier si un `...` précède les arguments abrégés : après lui, **plus d'appariement partiel**.

Protocole « maîtriser le moment de l'évaluation » — 4 étapes.

1. Se demander **si** l'argument sera évalué : « *in some cases the argument will never be evaluated* ».
2. Se demander **où** : fourni → chez **l'appelant** ; défaut → dans **la fonction**.
3. Si le moment compte, **forcer la promesse** en nommant l'argument sur une ligne seule (idiome `label`).
4. **Ne jamais compter sur un effet de bord** d'argument : « *it is bad style ...* ».

● Common mistakes

Erreur	Correction
Croire <code>return()</code> obligatoire	la dernière expression est la valeur
Oublier la dernière ligne du corps	la fonction rend alors la valeur de l'assignation, invisiblement
Définir un opérateur sans guillemets	<code>%!</code> n'est pas un identifiant → <code>"%!" <- ...</code>
Choisir <code>\</code> comme caractère d'opérateur	« <i>not a convenient choice ... special problems</i> »
Croire les défauts limités à des constantes	« <i>defaults may be arbitrary expressions</i> »
Croire qu'une fonction modifie son argument	call-by-value — « <i>lost after exit</i> »
Utiliser <code><<-</code> sans savoir où il écrit	il remonte les enclos (fiche 310)
Abréger un nom d'argument dans du code durable	l'ambiguïté dépend de ce qui reste
Abréger après un <code>...</code>	« <i>partial matching is only applied to arguments that precede it</i> »
Croire les primitives soumises aux mêmes règles	elles « <i>typically ignore tags</i> » — sauf <code>log</code> , <code>round</code> , <code>signif</code> , <code>rep</code> , <code>seq.int</code>
Utiliser <code>foo(x = y)</code> pour assigner	« <i>there is no guarantee that the argument will ever be evaluated</i> »
Croire <code>foo(x <- y)</code> local	il modifie <code>x</code> chez l'appelant
Croire un défaut évalué à la définition	il l'est quand on en a besoin , dans la fonction
Croire qu'une promesse est réévaluée	« <i>A promise will only be forced once</i> »
Écrire <code>list(...)</code> par réflexe	cela force tous les arguments
Attendre d'une fonction locale qu'elle se voie elle-même	« <i>not inherited by called functions in higher evaluation frames</i> »
Utiliser <code>missing()</code> en croyant lire la valeur	« <i>does not force evaluation</i> »

Ultimate Review

Définir. `nom <- function(args) expression` · « *The value of the expression is the value returned* » · le corps est « *usually a grouped expression* » · un appel « *may occur anywhere a function call is legitimate* » · les fonctions du système sont écrites en R et « *do not differ materially from user written functions* ».

Formel et anonyme. Un argument formel est un **symbole**, `symbole = expression`, ou `...` · « *Generally functions are assigned to symbols but they don't need to be* » — sans nom, c'est une **fonction anonyme**, « *most frequently used as arguments to other functions such as the `apply` family or `outer`* » · « *Functions are first class objects in R* ».

Opérateur binaire. Un nom de la forme `%anything%` · **guillemets obligatoires** à la définition · `%*%` et `%o%` en sont des exemples · `\` est un mauvais choix.

Arguments. Nommés → **n'importe quel ordre** · on peut commencer en **positionnel** puis nommer · défauts par `arg = expression` · « *defaults may be arbitrary expressions, even involving other arguments to the same function* » · `missing()` dit si un formel n'a pas été apparié — **sans forcer l'évaluation**.

`...` · Sert à « *allow one function to pass on argument settings to another* » · `list(...)` « *evaluates all such arguments and returns them in a named list* » (il force) · `..1`, `..2`, `..n` rendent « *the n-th unmatched argument* » · tout actuel n'appariant aucun formel **tombe dans** `...`.

Appariement — trois passes. (1) **tags exacts**, un formel = un actuel, sinon **erreur** ; (2) **tags partiels**, « *It is an error to have multiple partial matches* », et **seulement avant un** `...` ; (3) **position**, le `...` prenant « *the remaining arguments, tagged or not* ». Reste non apparié → **erreur**. Exemple : `f(f=1, fo=2)` **illégal**, `f(f=1, fooey=2)` **légal**. Les primitives ignorent les tags — exceptions : `log`, `round`, `signif`, `rep`, `seq.int`. Outils : `match.arg`, `match.call`, `match.fun`, `pmatch`.

Assignations. « *any ordinary assignments done within the function are local and temporary and are lost after exit* » · **call-by-value** : « *Changing the value of a supplied argument within a function will not affect the value of the variable in the calling frame* » · pour sortir : `<<-` ou `assign()`.

Évaluation. Fournis → **cadre de l'appelant** ; défauts → **cadre de la fonction** · **Paresseuse** : « *Arguments are not evaluated until needed ... in some cases the argument will never be evaluated* » · d'où : « *it is bad style to use arguments to functions to cause side-effects* », et `foo(x <- y)` modifie `x` chez l'appelant.

Promesses. Trois fentes : **expression, environnement, valeur** · à l'appel, chaque formel reçoit une promesse (environnement = **celui de l'appelant** ; pour un défaut, **l'environnement local**) · **forcer** = évaluer l'expression dans son environnement · « *A promise will only be forced once* » · `substitute` lit l'expression · les promesses **se forcent récursivement**.

L'idiome du forçage. Une ligne portant le seul nom de l'argument **fixe le moment de l'évaluation** :

```
function(x, label = deparse(x)) { label; x <- x + 1; print(label) } .
```

Les trois exemples avancés. `bdeff` — `as.factor` en garde-fou, `as.vector` pour ôter `dim`, produit de diagonales **par recyclage**, **SVD plutôt que valeurs propres**, résultat en **liste**. `no.dimnames` — « *removing the `dimnames` attribute will not achieve this effect* », il faut **des chaînes vides** ; `d[[1 <- 1 + 1]]` exploite l'assignation-expression ; « *effective and useful user functions can be quite short* ». `area` — récursive, `fun1` **locale**, passée **en argument à elle-même** parce que « *not inherited by called functions in higher evaluation frames* » ; « *a heavy overhead* ».

Active Recall

« A function is defined by **an assignment of the form** `nom <- function(arg_1, arg_2, ...)` `expression` . The `expression` is an R expression, (**usually a grouped expression**), that uses the arguments to calculate a value. **The value of the expression is the value returned for the function.** » (§10)

Il n'y a pas de `return` obligatoire : la valeur du corps est celle de **sa dernière expression** (fiche 308, §9.1). `return()` ne sert qu'à **sortir tôt**, comme dans l'exemple `area` du §10.6.3.

L'erreur classique est de terminer le corps par **une assignation** : la fonction rend alors cette valeur **invisiblement**, et l'appel semble ne rien produire.

« **Generally functions are assigned to symbols but they don't need to be.** The value returned by the call to `function` **is a function.** If this is not given a name it is referred to **as an anonymous function.** Anonymous functions are most frequently used as **arguments to other functions such as the `apply` family or `outer`** . » (R Language Definition §4.1.1)

L'exemple le plus visible du cours est en fiche 305 :

```
z <- outer(x, y, function(x, y) cos(y)/(1 + x^2))
```

C'est possible parce que « **Functions are first class objects in R. They can be used anywhere that an R object is required ... passed as arguments ... and returned as values from functions** » (§4.2).

« Had we given the function a name of the form `%anything%`, it could have been used as a binary operator » (§10.2) :

```
"%!%" <- function(X, y) { ... }  
X %!% y
```

« (Note the use of quote marks.) »

Les guillemets sont nécessaires parce que `%!%` n'est pas un identifiant valide (fiche 300) : le parseur ne peut pas le lire comme un nom. La chaîne contourne la restriction — c'est le « *objects can have names that are not identifiers* » de la fiche 302.

Et cela fonctionne parce qu'un **opérateur est une fonction** (fiche 301, §3.1.4) : `%*%` et `%o%` sont « *other examples of binary operators defined in this way* ».

« **The backslash symbol itself is not a convenient choice** as it presents special problems in this context. »

« This is done by a **three-pass process** » (R Language Definition §4.3.2) :

1. **Tags exacts.** « For each named supplied argument the list of formal arguments is searched for an item whose name **matches exactly. It is an error to have the same formal argument match several actuals or vice versa.** »
2. **Tags partiels.** « Each remaining named supplied argument is compared to the remaining formal arguments **using partial matching ... It is an error to have multiple partial matches. ... If the formal arguments contain ... then partial matching is only applied to arguments that precede it.** »
3. **Position.** « Any unmatched formal arguments are bound to unnamed supplied arguments, **in order. If there is a ... argument, it will take up the remaining arguments, tagged or not.** »

« *If any arguments remain unmatched an error is declared.* »

« Notice that if `f <- function(fumble, fooey) fbody`, then `f(f = 1, fo = 2)` is illegal, even though the 2nd actual argument only matches `fooey`. `f(f = 1, fooey = 2)` is legal though, since the second argument matches exactly and is removed from consideration for partial matching. » (§4.3.2)

Le premier cas. Passe 1 : aucun tag exact. Passe 2 : `f` est un préfixe de `fumble` et de `fooey` → **appariement partiel multiple** → erreur. Le fait que `fo` lèverait l'ambiguïté n'y change rien : `f` est traité **dans la même passe**, sur **tous** les formels restants.

Le second cas. Passe 1 : `fooey` apparie exactement et **sort du jeu**. Passe 2 : `f` n'a plus que `fumble` en face → unique → succès.

La leçon : l'ambiguïté d'un nom abrégé **dépend de ce qui reste**. Un appel qui marchait peut cesser de marcher parce qu'un **autre** argument a changé.

Non. « *Note that any ordinary assignments done within the function are local and temporary and are lost after exit from the function. Thus the assignment `X <- qr(X)` does not affect the value of the argument in the calling program.* » (§10.5)

Et la formulation générale (R Language Definition §4.3.3) : « *The semantics of invoking a function in R are call-by-value. ... Changing the value of a supplied argument within a function will not affect the value of the variable in the calling frame.* »

C'est la sémantique de copie de la fiche 302 (mécanisme ``*tmp*``). **L'unique exception du langage** reste **l'environnement**, « *not copied when passed to functions* » (fiche 303).

Pour sortir volontairement : *« either the "superassignment" operator, `<<-`, or the function `assign()` can be used »*.

« One of the most important things to know ... is that **supplied arguments and default arguments are treated differently**. The supplied arguments to a function are evaluated in the evaluation frame of the calling function. The default arguments to a function are evaluated in the evaluation frame of the function. » (§4.3.3)

	Évalué dans
argument fourni	le cadre de l'appelant
argument par défaut	le cadre de la fonction

C'est ce qui rend possible la phrase du §10.3 : « *defaults may be arbitrary expressions, even involving other arguments to the same function* ». Un défaut `n = length(x)` a besoin d'être évalué là où `x` existe — c'est-à-dire **dans la fonction**.

« *R has a form of lazy evaluation of function arguments. Arguments are not evaluated until needed. It is important to realize that in some cases the argument will never be evaluated.* » (§4.3.3)

La règle de style : « *Thus, it is bad style to use arguments to functions to cause side-effects. While in C it is common to use the form `foo(x = y)` ... this same style should not be used in R. There is no guarantee that the argument will ever be evaluated and hence the assignment may not take place.* »

Et un second avertissement : « *the effect of `foo(x <- y)`, if the argument is evaluated, is to change the value of `x` in the calling environment and not in the evaluation environment of `foo`* ».

Conséquence pratique utile : un argument jamais utilisé peut être **invalide sans conséquence** — un défaut coûteux n'est calculé que si on s'en sert.

« *When a function is called, each formal argument is assigned a promise in the local environment of the call, with the expression slot containing the actual argument and the environment slot containing the environment of the caller. If no actual argument ... is given and there is a default expression, it is similarly assigned ... but with the environment set to the local environment.* » (§4.3.3)

« *The process of filling the value slot of a promise by evaluating the contents of the expression slot in the promise's environment is called forcing the promise. A promise will only be forced once, the value slot content being used directly later on.* »

Trois fentes : expression, environnement, valeur. **Quatre temps** : créer, attendre, forcer, mémoriser.

« **Access to the unevaluated expression is also available using** `substitute` . » — c'est ainsi que `plot(x, y)` sait écrire « x » et « y » sur les axes (fiche 316).

« **When forcing a promise, other promises in its expression will also be forced recursively.** »

```
function(x, label = deparse(x)) {  
  label  
  x <- x + 1  
  print(label)  
}
```

« *This is occasionally useful when **a default expression depends on the value of another formal argument** ... This is seen in the following example **where the lone** `label` **ensures that the label is based on the value of** `x` **before it is changed in the next line.*** » (§4.3.3)

Sans cette ligne, la promesse de `label` ne serait forcée qu'au `print` , donc **après** `x <- x + 1` : `deparse(x)` décrirait la valeur **modifiée**.

La ligne isolée **ne calcule rien d'utile** — elle **fixe le moment de l'évaluation**. C'est l'idiome du « forçage » : nommer un argument sur une ligne seule.

« *Another frequent requirement is **to allow one function to pass on argument settings to another** ... This can be done by **including an extra argument, literally** `...` , **of the function, which may then be passed on.*** » (§10.4)

```
fun1 <- function(data, data.frame, graph = TRUE, limit = 20, ...) {  
  if (graph) par(pch = "*", ...)  
}
```

« *Less frequently, a function will need to refer to components of* `...` . **The expression** `List(...)` **evaluates all such arguments and returns them in a named list, while** `..1` ,

`..2` , etc. evaluate them one at a time, with `..n` returning the *n*-th unmatched argument. »

`list(...)` **force les promesses** (§2.1.9 : « *with promises being forced!* »). Tant qu'on **transmet** `...` , rien n'est évalué.

Et : « *If a function has `...` as a formal argument then any actual arguments that do not match a formal argument are matched with `...`* » (§2.1.9).

Parce que le cours prévient (§10.6.3) : « *Functions may be recursive, and may themselves define functions within themselves. Note, however, that such functions, or indeed variables, are not inherited by called functions in higher evaluation frames as they would be if they were on the search path.* »

`fun1` est **définie à l'intérieur de** `area` , donc locale. Les cadres d'évaluation créés par ses propres appels **ne la trouvent pas** par le chemin de recherche (fiche 306) — le chemin ne contient que `.GlobalEnv` , les objets attachés et les paquets.

La solution retenue par le cours est de **la passer en argument** : `area` termine par `fun1(f, a, b, fa, fb, a0, eps, lim, fun1)` — elle se transmet elle-même sous le nom `fun` .

C'est ce que le cours appelle « *a little puzzle in R programming* ». La fiche 310 montrera l'autre voie : la **portée lexicale**, qui rend cette gymnastique inutile dans d'autres constructions.

« *Removing the `dimnames` attribute will not achieve this effect, but rather the array must be given a `dimnames` attribute consisting of empty strings.* » (§10.6.2)

Sans `dimnames` , R **réinvente** des étiquettes à l'affichage — `[1,]` , `[,1]` , etc. Pour n'avoir **rien**, il faut des noms qui existent **et soient vides** :

```
dimnames(temp) <- list(rep("", nrow(X)), rep("", ncol(X)))
```

La fonction `no.dimnames()` du cours généralise à un nombre quelconque de dimensions en bouclant sur `dim(a)` , et le cours souligne qu'elle « *illustrates how **some effective and*** »

useful user functions can be quite short ».

Noter la ligne `d[[1 <- 1 + 1]] <- rep("", i)` : elle **incrmente et utilise** en une fois, parce qu'« *an assignment is an expression whose result is the value assigned* » (fiche 308).

`as.vector` : `table()` rend **un tableau à une dimension** (fiche 302), donc muni d'un `dim`. Le commentaire du cours est explicite — « *remove dim attr* ». Sans cela, l'arithmétique `1/sqrt(K) * N * ...` se heurterait à la règle 3 du recyclage mixte (fiche 305) : « *the arrays must all have the same dim attribute or an error results* ».

La SVD : « *It is numerically slightly better to work with the singular value decomposition on this occasion rather than the eigenvalue routines.* »

Puisque $E = I_v - A^T A$, les valeurs propres de E valent $1 - d_i^2$, les d_i étant **les valeurs singulières de A** — d'où `1 - sv$d^2`. Passer par $A^T A$ élèverait le **conditionnement au carré** ; la SVD travaille directement sur A .

Et le résultat est **une liste** : « *not only the efficiency factors as the first component, but also the block and variety canonical contrasts* ».

Flashcards

Question	Réponse
La forme d'une définition ?	<code>nom <- fonction(args) expression</code>
Qu'est-ce qui est renvoyé ?	La valeur de l'expression — la dernière
<code>return()</code> est-il obligatoire ?	Non
Les fonctions du système sont écrites en quoi ?	En R — elles « <i>do not differ materially</i> »
Les trois formes d'un argument formel ?	Un symbole · <code>symbole = expression</code> · <code>...</code>
Une fonction sans nom s'appelle ?	Une fonction anonyme
Où les rencontre-t-on surtout ?	La famille <code>apply</code> et <code>outer</code>
Les fonctions sont-elles de première classe ?	Oui — passables et renvoyables
Comment nommer un opérateur binaire ?	<code>%anything%</code>
Que faut-il à la définition ?	Des guillemets
Pourquoi ?	<code>%!%</code> n'est pas un identifiant
Deux opérateurs définis ainsi ?	<code>%%</code> et <code>%o%</code>
Quel caractère éviter ?	<code>\</code> — « <i>special problems</i> »
Les arguments nommés doivent-ils être ordonnés ?	Non
Peut-on mêler positionnel et nommé ?	Oui — positionnel d'abord
Un défaut peut-il être une expression ?	Oui — « <i>arbitrary expressions</i> »
Peut-il dépendre d'un autre argument ?	Oui
Quelle fonction dit qu'un argument manque ?	<code>missing()</code>
Force-t-elle l'évaluation ?	Non
À quoi sert <code>...</code> ?	Transmettre des arguments à une autre fonction
Comment tous les récupérer ?	<code>list(...)</code> — une liste nommée

Question	Réponse
Effet secondaire de <code>list(...)</code> ?	Il force toutes les promesses
Comment prendre le n-ième ?	<code>..n</code>
Que devient un actuel n'appariant aucun formel ?	Il tombe dans <code>...</code>
Combien de passes d'appariement ?	Trois
Passe 1 ?	Tags exacts
Passe 2 ?	Tags partiels
Passe 3 ?	Position
Que se passe-t-il s'il reste un actuel ?	Erreur
Deux appariements partiels possibles ?	Erreur
L'appariement partiel après un <code>...</code> ?	N'a pas lieu
Que prend le <code>...</code> en passe 3 ?	Tout le reste, nommé ou non
<code>f(f=1, fo=2)</code> pour <code>function(fumble, foey)</code> ?	Illégal
<code>f(f=1, foey=2)</code> ?	Légal
Pourquoi ?	<code>foey</code> sort du jeu en passe 1
Les primitives suivent-elles ces règles ?	Non — elles ignorent les tags
Cinq exceptions citées ?	<code>log</code> · <code>round</code> · <code>signif</code> · <code>rep</code> · <code>seq.int</code>
Trois fonctions d'appariement ?	<code>match.arg</code> · <code>match.call</code> · <code>match.fun</code>
L'accès à l'algorithme partiel ?	<code>pmatch</code>
Que deviennent les assignations dans une fonction ?	Locales et temporaires , perdues à la sortie
Quelle est la sémantique d'appel ?	Call-by-value
Une fonction modifie-t-elle son argument ?	Non
Comment assigner à l'extérieur ?	<code><<-</code> ou <code>assign()</code>

Question	Réponse
Où est évalué un argument fourni ?	Chez l'appelant
Où est évalué un défaut ?	Dans la fonction
Qu'est-ce que l'évaluation paresseuse ?	« <i>not evaluated until needed</i> »
Un argument peut-il n'être jamais évalué ?	Oui
Que dit le manuel des effets de bord d'arguments ?	« <i>it is bad style</i> »
Où <code>foo(x <- y)</code> modifie-t-il <code>x</code> ?	Dans l'environnement de l'appelant
Les trois fentes d'une promesse ?	Expression, environnement, valeur
Quel environnement pour un argument fourni ?	Celui de l'appelant
Et pour un défaut ?	L'environnement local
Que veut dire « forcer » ?	Évaluer l'expression dans son environnement
Combien de fois une promesse est-elle forcée ?	Une seule
Comment lire l'expression non évaluée ?	<code>substitute()</code>
Les promesses imbriquées ?	Forcées récurivement
Comment forcer volontairement ?	Nommer l'argument sur une ligne seule
Une fonction locale est-elle visible de ses appels ?	Non — pas par le chemin de recherche
Comment <code>area</code> contourne-t-elle cela ?	En se passant elle-même en argument
Pourquoi <code>as.vector(table(x))</code> dans <code>bdeff</code> ?	Pour ôter l'attribut <code>dim</code>
Pourquoi la SVD dans <code>bdeff</code> ?	« <i>numerically slightly better</i> »
Que rend <code>bdeff</code> ?	Une liste — efficacités et contrastes canoniques
Suffit-il d'ôter <code>dimnames</code> pour un affichage nu ?	Non — il faut des chaînes vides
Que dit le cours de <code>no.dimnames</code> ?	Que de bonnes fonctions peuvent être très courtes
Quel est le coût de <code>area</code> ?	« <i>a heavy overhead</i> »